PERTEMUAN 6 · Praktik terbimbing

Context-Free Grammar: BNF/EBNF, Derivasi, dan Pohon Sintaks

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 03 — Merancang tata bahasa (*grammar*) bebas konteks untuk bahasa sederhana.

Sub-CPMK yang dinilai pada pertemuan ini

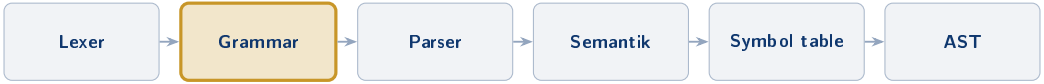
- **Sub-CPMK 3.1:** Membuat berkas grammar BNF dan EBNF untuk subset bahasa C, serta menguji derivasi dan pohon sintaks yang dihasilkan.
- **Sub-CPMK 3.4:** Membuat berkas grammar bebas konteks untuk subset bahasa C yang mencakup deklarasi, ekspresi, dan pernyataan, sebagai dasar parser dan proyek akhir.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
30	Ceramah: CFG, BNF, dan bahasa yang dibangkitkan
30	Latihan: konversi EBNF ke BNF
40	Praktik: menyusun grammar deklarasi, ekspresi, pernyataan
30	Praktik: lima derivasi dan dua pohon urai
20	Peer review berkas grammar dan penutup

Porsi praktik 120 menit. Keluaran pertemuan adalah berkas grammar dan bukti derivasi/pohon, bukan kode parser.

Posisi Pertemuan pada Rantai Proyek



- Pertemuan ini membangun simpul **grammar**: aturan yang menentukan bentuk program Subset-C.
- Grammar menjadi masukan langsung untuk analisis sintaksis, pembersihan rekursi kiri, dan parser pada Pertemuan 7–9.
- Berkas EBNF/BNF adalah fondasi proyek akhir.

CFG dan Notasi BNF

Context-Free Grammar (CFG)

$G = (V, T, P, S)$, dengan V himpunan nonterminal, T himpunan terminal, P aturan $A \to \alpha$ ($A \in V$, $\alpha \in (V \cup T)^*$), dan $S \in V$ simbol awal.

- Aturan mengganti satu nonterminal tanpa memedulikan konteks.
- BNF menulis alternatif dengan |: $A \to \alpha_1 \mid \alpha_2$.
- Nonterminal ditulis miring; terminal dalam tanda kutip atau huruf mesin.
- Produksi ε ditulis eksplisit.

Diagram Sintaks



- Diagram untuk $E \to T \mid T + E$.
- Jalur langsung T ke E menyatakan bagian + `E` opsional.
- Informasi sama dengan BNF, tetapi pilihan terlihat visual.

Contoh CFG Aritmetika

$$E \to E + T \mid T, \quad T \to T * F \mid F, \quad F \to (E) \mid \texttt{id}.$$

- E, T, dan F menyatakan ekspresi, term, dan faktor.
- Terminal: `id`, `+`, `*`, `(`, `)`.
- Bahasa yang dibangkitkan: $L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$.
- Contoh lain: $S \to aSb \mid \varepsilon$ membangkitkan $\{a^n b^n \mid n \geq 0\}$.

CFG tidak memeriksa tipe atau deklarasi; pemeriksaan makna dilakukan pada fase semantik.

Mengapa CFG Cocok untuk Sintaksis

1. Dapat menyatakan struktur bersarang: kurung, blok, dan pemanggilan fungsi.
2. Dapat diurai secara efisien dengan parser LL atau LR.
3. Memiliki teori matang untuk analisis dan transformasi.

Batas CFG

CFG memastikan bentuk program benar, tetapi tidak dapat memeriksa apakah variabel sudah dideklarasikan atau apakah tipe operand cocok. Pemeriksaan itu memerlukan tabel simbol dan aturan semantik.

Notasi EBNF

Notasi	Arti	Contoh
$X \mid Y$	pilihan	`int	float`
$[X]$	opsional	`[else stmt]`
$\{X\}$	nol atau lebih	`{stmt}`
(X)	pengelompokan	`(a | b) c`

- EBNF lebih ringkas dan mudah dibaca.
- Parser generator seperti Bison memakai BNF.
- EBNF dikonversi ke BNF sebelum implementasi.

Konversi EBNF ke BNF

1. Pilihan $X \mid Y$ menjadi dua aturan terpisah.
2. Opsional $[X]$ menjadi X atau ε.
3. Pengulangan $\{X\}$ menjadi nonterminal bantu rekursif.
4. Pengelompokan (X) menjadi nonterminal baru bila perlu.

Contoh

`params = param {"," param}` menjadi
params → param params_tail, params_tail → `","` `param` params_tail | ε.
`returnstmt = "return" [expr] ";"` menjadi
returnstmt → `"return"` opt_expr `";"`, opt_expr → expr | ε.

Berkas grammar.ebnf

```
// Berkas: spec/grammar.ebnf
// Tata bahasa EBNF Subset-C v0.4 (Bab VIII).

program      = { topdecl } EOF ;
topdecl      = type ID "(" [ params ] ")" ( block | ";" ) ;
params       = param { "," param } ;
param        = type ID ;
type         = "int" | "float" | "char" | "void" ;
block        = "{" { stmt } "}" ;
stmt         = decl ";" | block | ifstmt | whilestmt
     | forstmt
               | returnstmt | breakstmt | continuestmt
               | expr ";" | printstmt | ";" ;
decl         = type ID [ "=" expr ] ;
ifstmt       = "if" "(" expr ")" stmt [ "else" stmt ]
     ;
```

Listing 1: Bagian awal grammar.ebnf

- {} untuk daftar, [] untuk bagian opsional.
- Satu aturan per konstruksi.
- Nama nonterminal mencerminkan peran.
- Notasi ringkas untuk dibaca manusia.

Berkas ini harus dikonversi sebelum dipakai parser generator.

Berkas grammar.bnf

```
topdecl_list    ::= topdecl topdecl_list | epsilon
topdecl         ::= type ID "(" opt_params ")"
      topdecl_tail
opt_params      ::= params | epsilon
params          ::= param params_tail
params_tail     ::= "," param params_tail | epsilon
param           ::= type ID
type            ::= "int" | "float" | "char" | "void"
topdecl_tail    ::= block | ";"
block           ::= "{" stmt_list "}"
stmt_list       ::= stmt stmt_list | epsilon
stmt            ::= decl ";" | block | ifstmt |
      whilestmt | forstmt
                  | returnstmt | breakstmt |
                    continuestmt
                  | expr ";" | printstmt | ";"
decl            ::= type ID opt_init
```

Listing 2: Hasil konversi pada grammar.bnf

- Simbol bantu berakhiran _tail.
- Bagian opsional memakai awalan opt_.
- Pengulangan menjadi nonterminal rekursif.
- Konvensi nama memudahkan penelusuran asal aturan.

Derivasi Leftmost

Untuk grammar aritmetika dan string $\texttt{id} + \texttt{id} * \texttt{id}$:

$$\begin{aligned} E &\Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow \texttt{id} + T \\ &\Rightarrow \texttt{id} + T * F \Rightarrow \texttt{id} + F * F \Rightarrow \texttt{id} + \texttt{id} * F \\ &\Rightarrow \texttt{id} + \texttt{id} * \texttt{id}. \end{aligned}$$

- Leftmost selalu mengganti nonterminal paling kiri.
- Rightmost mengganti nonterminal paling kanan.
- Pada grammar tidak ambigu, keduanya menghasilkan pohon yang sama.

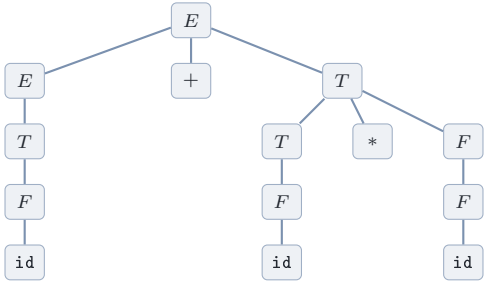
Bentuk Kalimat dan Pohon Penurunan

Bentuk kalimat (*sentential form*)

Setiap string yang muncul selama derivasi. Bentuk kalimat dapat memuat campuran terminal dan nonterminal; kalimat (*sentence*) hanya memuat terminal.

- Pada derivasi sebelumnya, $E + T * F$ adalah bentuk kalimat.
- $\texttt{id} + \texttt{id} * \texttt{id}$ adalah kalimat.
- Akar pohon adalah simbol awal; anak simpul mengikuti ruas kanan aturan.
- Daun dibaca kiri ke kanan membentuk string turunan.

Pohon Urai untuk `id + id * id`



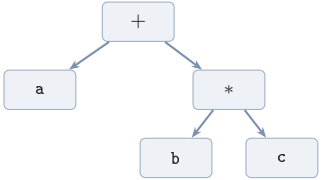
Struktur pohon menunjukkan bahwa perkalian lebih dalam daripada penjumlahan, sesuai prioritas operator.

Pohon Urai dibandingkan AST

Aspek	Pohon urai	AST
Isi	Semua simbol tata bahasa	Konstruksi bermakna
Ukuran	Lebih besar	Lebih ringkas
Detail	Kurung, titik koma, kata kunci	Hanya operand dan operator
Penggunaan	Analisis sintaksis	Analisis semantik dan kode

AST menghilangkan detail sintaksis yang tidak diperlukan fase berikutnya. Konversi dapat dilakukan saat parser melakukan *reduce*.

AST untuk `a + b * c`



AST ekspresi

Notasi prefix

```
(+ a (* b c))
```

- Tidak ada tanda kurung sintaksis.
- Struktur operator langsung terlihat.
- Akar + menunjukkan penjumlahan sebagai operasi terluar.

Hierarki Nonterminal Grammar Subset-C

Nonterminal	Tanggung jawab
`program`	Daftar deklarasi tingkat atas
`topdecl`	Deklarasi fungsi atau prototipe
`stmt`	Deklarasi lokal, blok, dan pernyataan kontrol
`decl`	Tipe, identifier, dan inisialisasi opsional
`expr`	Ekspresi; titik masuk ke tingkat prioritas
`assign`	Penugasan atau ekspresi logika
`logor–term`	Satu tingkat untuk satu kelompok operator
`primary`	Literal, identifier, panggilan, kurung

Satu nonterminal untuk satu tingkat prioritas; operator berprioritas lebih tinggi berada lebih dalam pohon.

Aturan Deklarasi dan Fungsi

$$
\begin{aligned}
\text{topdecl} &\rightarrow \texttt{type ID "("} \ \text{opt_params} \ \texttt{")"} \ \text{topdecl_tail},\\
\text{topdecl_tail} &\rightarrow \text{block} \mid \texttt{";"},\\
\text{opt_params} &\rightarrow \text{params} \mid \varepsilon,\\
\text{params} &\rightarrow \text{param params_tail},\\
\text{params_tail} &\rightarrow \texttt{","} \ \texttt{param} \ \text{params_tail} \mid \varepsilon,\\
\text{param} &\rightarrow \texttt{type ID}.
\end{aligned}
$$

- Fungsi dengan tubuh blok atau prototipe tanpa tubuh.
- Parameter opsional ditangani `opt_params`.
- Daftar parameter memakai rekursi kanan.

Aturan Pernyataan

$$\begin{aligned}\text{stmt} \rightarrow\ & \text{decl } \texttt{";"} \mid \text{block} \mid \text{ifstmt} \mid \text{whilestmt} \\ & \mid \text{forstmt} \mid \text{returnstmt} \mid \text{breakstmt} \mid \text{continuestmt} \\ & \mid \text{expr } \texttt{";"} \mid \text{printstmt} \mid \texttt{";"}.\end{aligned}$$

- Pemisahan ini memudahkan parser memilih cabang berdasarkan token awal.
- `if` langsung memilih `ifstmt`; `while` memilih `whilestmt`.
- Deklarasi lokal dan blok tetap berada dalam `stmt`.

Aturan Ekspresi Berprioritas

$$
\begin{aligned}
\text{expr} &\rightarrow \text{assign},\\
\text{assign} &\rightarrow \texttt{ID "="}\ \text{assign} \mid \text{logor},\\
\text{logor} &\rightarrow \text{logand}\ \text{logor_tail},\\
\text{logand} &\rightarrow \text{equality}\ \text{logand_tail},\\
\text{equality} &\rightarrow \text{relation}\ \text{equality_tail},\\
\text{relation} &\rightarrow \text{additive}\ \text{relation_tail},\\
\text{additive} &\rightarrow \text{term}\ \text{additive_tail},\\
\text{term} &\rightarrow \text{unary}\ \text{term_tail},\\
\text{unary} &\rightarrow \texttt{"!"}\ \text{unary} \mid \texttt{"-"}\ \text{unary} \mid \texttt{"+"}\ \text{unary} \mid \text{primary}.
\end{aligned}
$$

Prioritas naik dari `logor` ke `term`; unary dan primary berada paling dalam.

Contoh: Prioritas dan Tanda Kurung pada `arith.sc`

```
// Berkas: examples/arith.sc
int main() {
    int a = 2 + 3 * 4;
    int b = (2 + 3) * 4;
    print(a);
    print(b);
    return 0;
```

Listing 3: Program uji prioritas `arith.sc`

- `2 + 3 * 4`: akar $+$ dengan anak kanan $*$.
- `(2 + 3) * 4`: akar $*$ dengan anak kiri $+$.
- Perbedaan berasal dari tanda kurung dan tercermin pada pohon urai.
- Grammar harus menghasilkan struktur yang berbeda untuk kedua ekspresi.

Kegiatan Mahasiswa di Kelas: P2-A

Tata bahasa Subset-C

1. Lengkapi `spec/grammar.ebnf` untuk seluruh konstruksi.
2. Turunkan lima string: ekspresi, deklarasi, if, while, dan fungsi.
3. Konversi EBNF ke BNF pada `spec/grammar.bnf`.
4. Gambar pohon urai untuk dua string.
5. Tukar berkas dengan teman untuk peer review.

Periksa bahwa setiap aturan EBNF memiliki padanan BNF dan setiap nonterminal bantu diberi nama konsisten.

Tugas dan Artefak

Artefak Sub-CPMK 3.1 dan 3.4

Berkas grammar BNF/EBNF untuk deklarasi, ekspresi, dan pernyataan, disertai lima derivasi, dua pohon urai, dan catatan hasil uji.

- EBNF mencakup semua konstruksi yang didukung Subset-C.
- Lima derivasi benar dan terdokumentasi.
- BNF konsisten dengan EBNF; simbol bantu dinamai konsisten.
- Dua pohon urai benar dan sesuai derivasi.
- Catatan peer review lengkap.

P2-A menjadi bagian Praktikum 2 (12%) yang dilaporkan pada Pertemuan 9.

Rangkuman dan Jembatan

- CFG $G = (V, T, P, S)$ membangkitkan bahasa melalui derivasi $S \Rightarrow^* w$.
- BNF menulis aturan dengan alternatif; EBNF menambahkan pengulangan, opsional, dan pengelompokan.
- EBNF dikonversi ke BNF dengan aturan sistematis dan simbol bantu `_tail` serta `opt_`.
- Derivasi leftmost dan rightmost menghasilkan pohon urai; AST menyederhanakan detail sintaksis.
- Grammar Subset-C memisahkan deklarasi, pernyataan, dan tingkat prioritas ekspresi.

Pertemuan berikutnya

Pertemuan 7 membersihkan grammar dari ambiguitas, rekursi kiri, dan kebutuhan left factoring. Grammar yang bersih menjadi syarat parser top-down pada Pertemuan 9.